

OSWG

Oddly Specific Wordlist Generator

Version 0.7.0

Developer Guide

Repository: <https://github.com/dmarakom6/oswg>

Version: 0.7.0

License: MIT

Homebrew tap: [dmarakom6/homebrew-oswg](#)

Docker image: [ghcr.io/dmarakom6/oswg](#)

Generated: September 24, 2026

Official documentation - generated from the repository.

1 Architecture

OSWG is a single Python package with an embedded web front end. A **FastAPI** application in `src/oswg/` provides the REST API, serves the pre-built **SvelteKit** dashboard from `src/oswg/static`, and runs every job as an async task with **WebSocket** progress updates. Scraping uses `httpx` and `BeautifulSoup`; **JavaScript rendering**, interactive login, and session capture are delegated to **Playwright**.

Component	Role
FastAPI backend (<code>src/oswg/</code>)	REST endpoints, job orchestration, WebSocket updates.
SvelteKit UI (<code>ui/</code>)	Dashboard; the production build is copied into <code>src/oswg/static</code> and served by the backend.
SQLite store	Jobs and templates persisted via <code>aiosqlite</code> .
WebSocket	Real-time job progress pushed to the dashboard.
Playwright	JS rendering, interactive login, session capture.

1.1 Repository layout

```

oswg/
|-- src/oswg/
|   |-- __init__.py      package version
|   |-- cli.py           typer CLI: generate, scrape, mutate, test, ...
|   |-- launcher.py      uvicorn launcher for `oswg ui`
|   |-- database.py      SQLite job/template store
|   |-- models.py        Pydantic request/response models
|   |-- config.py        settings (XDG data dir, retention)
|   |-- core/
|       |-- scraper.py    crawler: httpx, robots, rate limit, JS
|       |-- generator.py  wordlist generation pipeline
|       |-- mutations.py  mutation engine
|       |-- presets.py    quick/standard/aggressive/extreme
|       |-- rulegen.py    jtr/hashcat rule emission
|       |-- ai.py         Ollama/OpenAI completions
|       |-- test_runner.py external cracker drivers
|       |-- session.py    Playwright login and sessions
|       |-- emails.py, usernames.py, wordlists.py, ...
|   |-- routers/         FastAPI routers (generate, scrape, jobs, ...)
|   |-- services/        job manager, template store
|   |-- static/          SvelteKit build output
|-- ui/                  SvelteKit frontend source
|-- tests/               pytest unit + e2e suites
|-- build_binary.py      PyInstaller build helper
|-- oswg.spec            PyInstaller spec
|-- Makefile             build/dev/lint/test/hooks targets
|-- pyproject.toml       packaging, ruff, pytest configuration
-- .github/workflows/    ci.yml, release.yml, release-binaries.yml

```

2 Development setup

Create a virtual environment, install the package in editable mode with the dev and js extras, and install the UI dependencies:

```
python -m venv venv
source venv/bin/activate
pip install -e '.[dev,js]'
cd ui
npm ci
cd ..
```

`make build` builds the SvelteKit front end and copies `ui/build` into `src/oswg/static`, which is

```
make build    # npm run build in ui/, then copy static assets
make dev      # make build + python -m oswg ui
```

`make build` is required before running the app or the E2E suite - the backend serves the dashboard from `src/oswg/static`, so an empty static directory means a blank UI.

3 Testing

The project uses `ruff` and `svelte-check` for linting, plus `pytest` for a unit suite and a Playwright E2E

Command	What it runs
<code>make lint</code>	<code>ruff check src/ tests/</code> and <code>npx svelte-check</code> in <code>ui/</code> .
<code>pytest -q</code>	The 125 unit tests; E2E tests are deselected by default (<code>pytest</code> adopts <code>-m 'not e2e'</code>).
<code>pytest -m e2e</code>	The Playwright E2E suite in <code>tests/e2e/</code> , which boots the app plus a fixture site.
<code>make hooks</code>	Enable the pre-push git hook (<code>git config core.hooksPath .githubhooks</code>).

The E2E suite requires `make build` first (the backend serves the built UI) and Playwright Chromium installed (`playwright install chromium`). The pre-push hook (`.githubhooks/pre-push`) runs `ruff`, the unit tests, and `svelte-check` before every push.

4 CI/CD

Three GitHub Actions workflows automate testing, releasing, and publishing. `ci.yml` runs 3 jobs:

Workflow	Trigger	Jobs / outputs
<code>ci.yml</code>	Every push and pull request	backend (<code>ruff</code> + <code>pytest</code> unit), UI (<code>svelte-check</code> + <code>build</code>), E2E (Playwright).
<code>release.yml</code>	Push to master	Test job, then <code>python-semantic-release</code> versions the package and tags <code>vX.Y.Z</code> .
<code>release-binaries.yml</code>	<code>v*</code> tags (or <code>workflow_dispatch</code>)	Binaries, PyPI, <code>.deb</code> , Homebrew bump, Docker image, GitHub Release.

`release-binaries.yml` builds PyInstaller binaries for `linux-x86_64`, `macos-x86_64`, and `macos-arm64`; publishes the wheel and sdist to PyPI; builds a `.deb` and attaches it to the release; bumps the Homebrew formula in the `dmarakom6/homebrew-oswg` tap; and builds and pushes the multi-arch (`linux/amd64` and `linux/arm64`) Docker image to `ghcr.io/dmarakom6/oswg`.

5 Release flow

Releases are fully automated once a version is chosen:

- Commit the version bump with a conventional commit: ``chore(release): X.Y.Z``.
- Push to master. ``release.yml`` runs its test job, then `semantic-release` creates the ``vX.Y.Z`` tag.

Pushing the `vX.Y.Z` tag triggers `release-binaries.yml`, which produces the binaries, the `.deb`, the PyPI package, the Homebrew bump, and the Docker image.

- The GitHub Release is created with the binaries and the `.deb` attached automatically.

Step	Trigger	Workflow	Outputs
Version commit	chore(release): X.Y.Z on master	release.yml	vX.Y.Z tag
Tag build	vX.Y.Z tag pushed	release-binaries.yml	binaries, .deb, PyPI, Homebrew, Docker

The current documented version is 0.7.0. Version bumps flow through the semantic-release pipeline - never edit `pyproject.toml` or `src/oswg/__init__.py` by hand for a release.